



KCG
COLLEGE OF TECHNOLOGY
AFFILIATED TO ANNA UNIVERSITY | AUTONOMOUS

SECURE HETEROGENEOUS OVER-THE-AIR (OTA) FIRMWARE UPDATE MECHANISM FOR IOT DEVICES

PROJECT REPORT

Submitted by

RITESH V – 9123128039

RITHIK SHARMA A - 9123128040

PRIYANKA DEVI S - 9123128034

in the partial fulfilment for the award of the degree

of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)

KCG COLLEGE OF TECHNOLOGY

KARAPAKKAM , CHENNAI 600097

(AUTONOMOUS)



KCG
COLLEGE OF TECHNOLOGY
AFFILIATED TO ANNA UNIVERSITY | AUTONOMOUS

REG. NO : 9123128039,9123128040,9123128034

Name of the Course : PROJECT REPORT

It is to certify that this is a Bonafide record of the work carried out by Ritesh V (9123128039) , Rithik Sharma A (9123128040) Priyanka Devi S (9123128034) of VI Semester CSE – CYBER SECURITY Department , during the even semester of the academic year 2025 - 2026

Faculty In-Charge :

HOD :

INTERNAL EXAMINER:

DATE OF EXAMINATION :

1. Objective and Scope

1.1 Objective

The objective of this project is to design and develop SecureOTA IDE — a forked and extended version of the Arduino IDE that replaces the existing insecure OTA (Over-The-Air) upload mechanism with a fully encrypted, authenticated, and traceable firmware delivery pipeline. Unlike the stock Arduino OTA which transmits compiled binaries in plaintext over UDP/TCP with no authentication, SecureOTA IDE encrypts the firmware directly inside the IDE before transmission and establishes a secure channel to the target device.

In addition to security enhancements, the IDE is extended with three new embedded panels: a Firmware Version History panel that tracks every upload made to each device, an Update Relevance Checker that evaluates whether the new sketch contains changes relevant to the device's designated function, and a Device Dashboard that displays the real-time status, IP address, current firmware version, and health metrics of all discovered network devices — all from within the IDE itself

1.2 Scope

The scope of this project covers the following domains:

In Scope

- Cloning the Arduino IDE 2.x codebase (open-source, built on Eclipse Theia / Electron / TypeScript).
- Replacing the standard ArduinoOTA upload pathway with an encrypted equivalent — AES-256 payload encryption over a TLS 1.3 secured TCP channel, initiated directly from the IDE's Upload button.
- Extending the Arduino OTA library on the device side (ESP32/Arduino) to accept and decrypt the encrypted firmware stream.
- Adding a Firmware Version History panel inside the IDE that logs each upload (sketch name, version tag, timestamp, target device, upload result).
- Adding an Update Relevance Checker panel that compares the current sketch's tagged functionality against the device's registered function profile.
- Adding a Device Dashboard panel that lists discovered OTA-capable devices on the local network with live status.
- All additions are self-contained IDE plugins/extensions — the core editor, compiler, and Board Manager functionality of Arduino IDE remains unchanged.

Out of Scope

- Cloud-based or remote OTA (the secure channel is always IDE → Device directly, over the local network or direct Wi-Fi).
- Support for non-OTA upload methods (USB Serial upload is untouched).
- Modifying the Arduino compiler toolchain (avr-gcc, xtensa-esp32-elf-gcc) or build system.
- Cellular, LoRa, or Bluetooth-based OTA delivery.

2. Requirements Analysis

2.1 Functional Requirements

Req. ID	Requirement	Priority
FR-01	The IDE shall compile sketches using the existing Arduino build system and produce a binary (.bin) identical to stock Arduino IDE.	High
FR-02	The IDE shall encrypt the compiled firmware binary using AES-256-CBC with a per-session key before initiating an OTA upload.	High
FR-03	The IDE shall establish a TLS 1.3 encrypted TCP channel to the target device for all OTA firmware transfers.	High
FR-04	The device-side OTA library (modified ArduinoOTA) shall decrypt the received firmware before writing to flash.	High
FR-05	The IDE shall verify the firmware integrity using SHA-256 hash comparison after transfer completion.	High
FR-06	The IDE shall digitally sign the firmware using ECDSA and embed the server public key on the device for verification.	High
FR-07	The Firmware Version History panel shall record every OTA upload: sketch name, version, device IP, timestamp, and result.	High
FR-08	The IDE shall allow users to assign a version tag and function tags to each sketch before upload.	Medium
FR-09	The Update Relevance Checker shall compare sketch function tags against the device's registered function profile and display a relevance score.	Medium
FR-10	The Device Dashboard shall discover all SecureOTA-capable devices on the local network via mDNS/UDP broadcast.	High

Req. ID	Requirement	Priority
FR-11	The Device Dashboard shall display each device's IP, current firmware version, last-updated timestamp, and online/offline status.	High
FR-12	The IDE shall support rollback by re-uploading a previously recorded firmware version from the Version History panel.	Medium

2.2 Non-Functional Requirements

Req. ID	Category	Description
NFR-01	Security	Firmware must never be transmitted in plaintext; AES-256 encryption is mandatory for all OTA transfers.
NFR-02	Security	Tampered or unsigned firmware must be rejected by the device before writing to flash.
NFR-03	Compatibility	The IDE must retain 100% compatibility with all existing Arduino sketches, libraries, and boards.
NFR-04	Usability	The encrypted OTA upload must require no extra steps from the user compared to standard Arduino OTA — same one-click Upload.
NFR-05	Reliability	A failed OTA transfer must never leave the device in an unbootable state (dual-partition + rollback on ESP32).
NFR-06	Performance	Encryption overhead must not increase OTA upload time by more than 20% compared to unencrypted Arduino OTA.
NFR-07	Portability	The IDE extensions must build on Windows, macOS, and Linux (matching Arduino IDE 2.x platform support).

2.3 Hardware Requirements

- Development machine: PC / laptop running Windows 10+, macOS 12+, or Ubuntu 20.04+ (for IDE development and usage).
- Target device: ESP32 (recommended — hardware AES accelerator, dual OTA partitions, native TLS via mbedTLS).
- Local Wi-Fi network connecting the development machine and ESP32.

2.4 Software Requirements

- Arduino IDE 2.x source code (open-source, GitHub: [arduino/arduino-ide](#)) — base for the fork.
- Eclipse Theia framework (TypeScript/Node.js) — IDE UI shell.
- Electron — desktop application wrapper.
- Arduino CLI (Go) — used internally by the IDE for compilation and board management.
- Modified ArduinoOTA library (C++) — extended to support encrypted reception.
- mbedTLS (already present on ESP32 via ESP-IDF) — for AES-256, SHA-256, ECDSA, and TLS on the device.
- Node.js crypto module / Python cryptography library — for IDE-side encryption and signing.

3. Process and Detailed Design

3.1 High-level Process: How Existing Arduino OTA Works

In the standard Arduino IDE, when a user selects a network port (discovered via mDNS) and clicks Upload, the IDE invokes the arduino-cli which calls a Python-based OTA upload script (espot.py for ESP32 or similar). This script sends the compiled .bin file over a plain UDP/TCP socket to the ArduinoOTA listener running on the device. The device writes the received binary directly to a secondary flash partition and reboots. There is no encryption, no authentication, no integrity check, and no record of the upload.

3.2 Proposed Process: SecureOTA IDE Upload Flow

The SecureOTA IDE replaces the upload script pathway with a new Secure Upload Pipeline while keeping every other IDE action identical.

The user experience is unchanged: select a network port → click Upload. Behind the scenes, the following process runs:

Step	Actor	Action	Change from Standard Arduino OTA
1	Arduino CLI (IDE)	Sketch compiled to .bin using existing toolchain.	No change — identical to stock IDE.
2	SecureOTA Upload Engine (IDE)	SHA-256 hash of .bin computed. ECDSA signature generated using IDE's private key.	NEW — stock IDE does no signing.
3	SecureOTA Upload Engine (IDE)	AES-256-CBC session key + IV generated. .bin encrypted. Package assembled: [encrypted_bin hash signature metadata].	NEW — stock IDE sends plaintext .bin.
4	SecureOTA Upload Engine (IDE)	TLS 1.3 TCP connection established to device on port 3232 (same port as Arduino OTA, now with TLS).	NEW — stock IDE uses plain TCP/UDP.

Step	Actor	Action	Change from Standard Arduino OTA
5	SecureOTA Upload Engine (IDE)	Encrypted package transmitted in chunks over TLS channel. Session AES key sent encrypted with device's public key.	NEW — stock IDE sends raw binary.
6	Modified ArduinoOTA Library (Device)	Device receives chunks, decrypts AES session key using its stored private key, decrypts firmware.	NEW — stock ArduinoOTA writes directly.
7	Modified ArduinoOTA Library (Device)	SHA-256 hash verified. ECDSA signature verified using embedded IDE public key.	NEW — stock ArduinoOTA has no verification.
8	Modified ArduinoOTA Library (Device)	Firmware written to standby partition. Device reboots into new firmware.	Same partition mechanism, now secured.
9	SecureOTA IDE — Version History Panel	Upload event logged: sketch name, version tag, function tags, device IP, timestamp, result.	NEW — stock IDE logs nothing.
10	SecureOTA IDE — Device Dashboard	Device dashboard refreshes to show new firmware version and last-updated time.	NEW — stock IDE has no dashboard.

3.3 Detailed Design — IDE-Side Components

3.3.1 SecureOTA Upload Engine

This is a new TypeScript/Node.js module integrated into the IDE's upload pipeline. It intercepts the upload command when a Network Port is selected (instead of a COM/USB port) and executes the secure upload sequence. It exposes a single asynchronous function: `secureUpload(binaryPath, deviceIP, devicePublicKey)` which performs all cryptographic operations and manages the TLS connection.

- Key management: The IDE maintains an ECDSA key pair in local storage (generated once on first launch). The IDE's public key is embedded into the device firmware during the initial USB flash provisioning step.
- Session key generation: A fresh AES-256 key and 128-bit IV are generated per upload using `Node.js crypto.randomBytes()`.
- The session key is RSA-OAEP or ECDH-encrypted with the device's public key so only the target device can decrypt it.

3.3.2 Firmware Version History Panel

A new IDE side panel (Theia view extension, TypeScript + React) that persists upload records in a local SQLite database (or JSON file) on the development machine. Each record stores:

- Sketch name and file path
- User-assigned version tag (e.g., v1.0.0, v1.2-beta)
- Function tags assigned to the sketch (e.g., temperature_sensor, actuator)
- Target device IP address and device name
- Upload timestamp
- Upload result (SUCCESS / FAILED / ROLLED_BACK)
- SHA-256 hash of the uploaded binary (for re-upload integrity)

The panel allows the user to re-upload any previous version from history (rollback), compare two versions side by side by sketch name/hash, and export the log as CSV.

3.3.3 Update Relevance Checker Panel

Before each OTA upload, the IDE displays a relevance assessment popup/panel. The user tags the current sketch with `function_tags` (what the sketch does). The device, during provisioning, is registered with its own `function_profile` (what it is used for). The Relevance Checker computes:

$$\text{relevance_score} = \frac{|\text{sketch_function_tags} \cap \text{device_function_profile}|}{|\text{device_function_profile}|}$$

The panel displays the score as a percentage, lists matched and unmatched tags, and warns the user if the score is below a threshold (e.g., below 30%). The user can proceed anyway (override) or cancel. This prevents accidentally uploading a lighting-control sketch to a temperature-monitoring device.

3.3.4 Device Dashboard Panel

A persistent side panel inside the IDE that continuously discovers SecureOTA-capable devices on the local network using mDNS queries (the same service discovery used by standard Arduino OTA, `_arduino._tcp`). Discovered devices that support the SecureOTA protocol advertise an additional TXT record: `secureota=true`. The panel displays a table of all discovered devices with: device name, IP address, current firmware version (reported by the device via a lightweight status broadcast), last-upload timestamp, online/offline status, and a one-click Upload to Device button.

3.4 Detailed Design — Device-Side Components

3.4.1 Modified ArduinoOTA Library

The existing ArduinoOTA library (C++ for ESP32/Arduino) is extended to support the SecureOTA protocol. The library is distributed as a drop-in replacement — the user includes `SecureArduinoOTA.h` instead of `ArduinoOTA.h` in their sketch, with identical API calls (`begin()`, `handle()`, `setPassword()` replaced by `setPublicKey()`).

- Listens on TCP port 3232 with TLS 1.3 (using ESP32 mbedTLS).
- On connection, performs TLS handshake using the device's certificate.
- Receives encrypted AES session key, decrypts it using device's stored private key.
- Receives firmware chunks, verifies per-chunk HMAC-SHA256.
- After full receipt: decrypts firmware, verifies SHA-256 hash, verifies ECDSA signature.
- On success: writes to standby partition, triggers restart.
- On failure: discards buffer, logs error, continues running current firmware.
- Broadcasts a status UDP packet every 30 seconds containing device name, current firmware version string, and IP (for the Device Dashboard to receive).

4. New Algorithms and Protocols Used

4.1 AES-256-CBC — Firmware Payload Encryption

Advanced Encryption Standard with a 256-bit key in Cipher Block Chaining (CBC) mode is used to encrypt the firmware binary before transmission. The server generates a fresh 256-bit session key and 128-bit Initialisation Vector (IV) for each OTA session. The session key is securely exchanged during the TLS handshake. CBC mode ensures that identical plaintext blocks produce different ciphertext, preventing pattern analysis.

Why AES-256: ESP32 includes a hardware AES accelerator (AES-NI equivalent), making AES-256 decryption efficient on-device without significant power overhead.

4.2 ECDSA with NIST P-256 — Firmware Digital Signature

Elliptic Curve Digital Signature Algorithm (ECDSA) using the NIST P-256 (secp256r1) curve is used to sign the firmware binary on the server and verify it on the device. ECDSA provides equivalent security to RSA-3072 with a much smaller key size (32 bytes for the private key), which is important for embedding the public key in the ESP32 flash.

Signature process: The server computes SHA-256(firmware_binary), then signs the digest with its ECDSA private key. The device embeds the server's ECDSA public key at compile time and uses it to verify the signature after decryption.

4.3 SHA-256 — Integrity Hash

SHA-256 is applied to the complete decrypted firmware binary after all chunks are received. The computed hash is compared against the hash transmitted in the firmware metadata package. This provides an end-to-end integrity check independent of the ECDSA signature, detecting any corruption during storage or transmission.

4.4 HMAC-SHA256 — Per-Chunk Integrity

Each transmitted firmware chunk (typically 4 KB) carries an HMAC-SHA256 tag computed using a chunk-specific key derived from the session key and chunk index. This detects data injection or corruption at the chunk level during multi-packet transfer, before the full binary is assembled.

4.5 Mutual TLS 1.3 (mTLS) — Channel Authentication

TLS 1.3 is configured on the MQTT broker with mutual authentication: both the server and each device present X.509 certificates. The server's CA signs device certificates at registration time. During the MQTT connection, the broker validates the device certificate; only registered devices with valid,

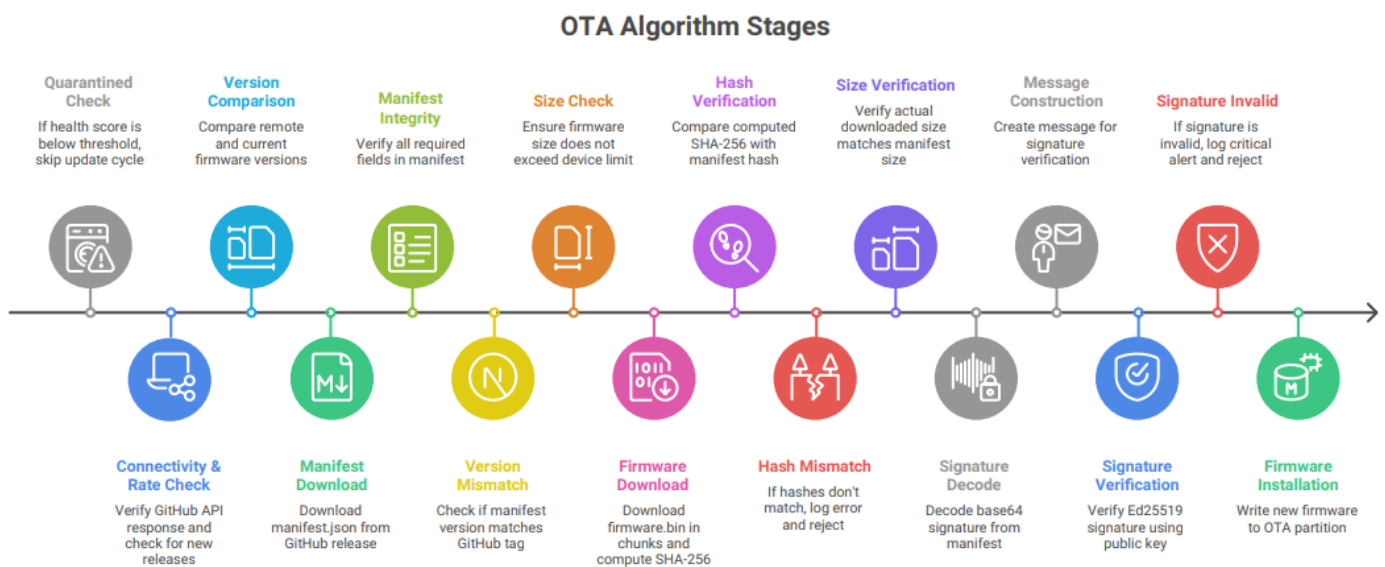
unrevoked certificates are permitted to connect. This prevents rogue servers from pushing malicious firmware and prevents unauthorised devices from pulling updates.

4.6 Update Relevance Scoring Algorithm

A lightweight tag-matching algorithm determines whether a firmware update is relevant to a specific device. Each firmware package carries a JSON array of `changelog_tags` (e.g., `["sensor_driver", "power_management", "wifi_stack"]`). Each device is registered with a `function_tags` array (e.g., `["temperature_sensor", "sensor_driver"]`). The relevance score is computed as:

$$\text{relevance_score} = |\text{changelog_tags} \cap \text{function_tags}| / |\text{function_tags}|$$

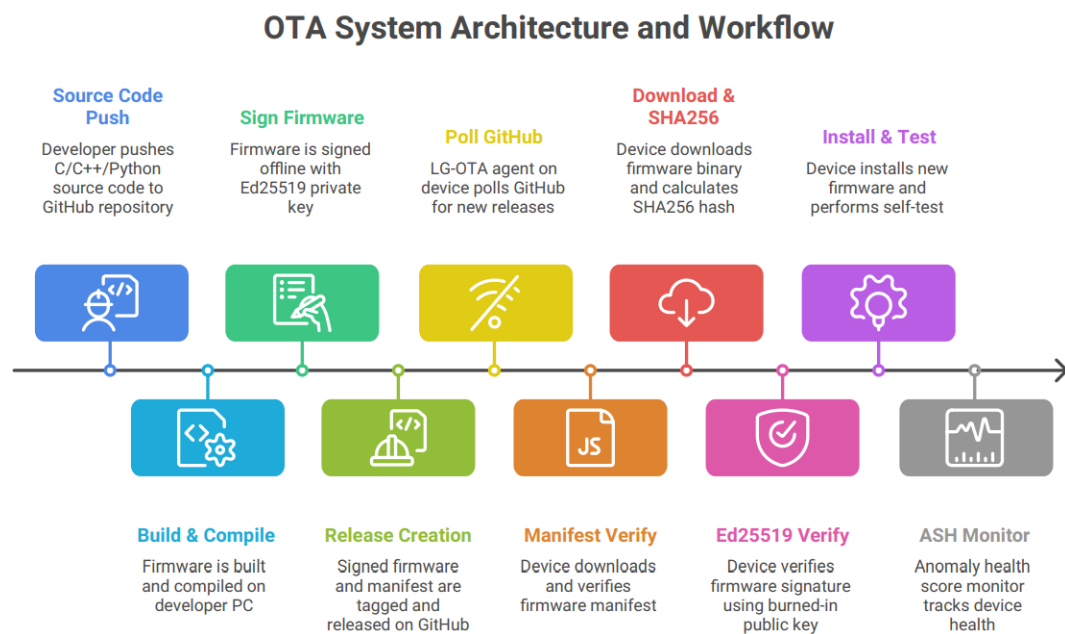
A score above a configurable threshold (default 0.3) flags the update as relevant. Administrators can override this decision from the dashboard.



5. System Architecture

5.1 Architecture Overview

SecureOTA IDE follows a two-tier client-device architecture. There is no cloud server or intermediary — the IDE on the developer's machine communicates directly with the target ESP32 device over the local Wi-Fi network, identical to standard Arduino OTA but with full encryption and authentication. The architecture is divided into two physical nodes (IDE Host Machine and ESP32 Device) and four logical layers.



5.2 Layered Architecture Diagram

IDE HOST MACHINE (Developer's PC / Laptop)	
UI Layer	Arduino IDE Clone (Theia/Electron): Code Editor, Serial Monitor, Board Manager, Port Selector — all unchanged from stock Arduino IDE.
Extension Layer	Three new IDE panels: Firmware Version History Panel, Update Relevance Checker Panel, Device Dashboard Panel — implemented as Theia view extensions (TypeScript + React).
Upload Engine	SecureOTA Upload Engine (TypeScript/Node.js): Intercepts network port upload, performs AES-256 encryption, ECDSA signing, TLS 1.3 connection setup, chunked transmission.

Key Store	Local key store: IDE ECDSA key pair, device public keys, upload history database (SQLite / JSON on local filesystem).
------------------	---

ESP32 DEVICE (Target Arduino / IoT Device)	
OTA Listener	Modified ArduinoOTA Library (SecureArduinoOTA.h): Listens on TCP 3232 with TLS 1.3, receives encrypted firmware chunks, verifies HMAC per chunk.
Crypto Layer	mbedTLS (ESP32 hardware-accelerated): ECDH session key decryption, AES-256-CBC firmware decryption, SHA-256 hash verify, ECDSA signature verify.
Flash Layer	ESP32 Dual OTA Partition Manager: Writes verified firmware to standby partition, switches boot partition on success, auto-rollbacks on boot failure.
Status Broadcaster	UDP Status Broadcast (every 30s): Sends device name, current firmware version, IP address — consumed by IDE Device Dashboard panel.

LOCAL NETWORK (Wi-Fi)	
mDNS Discovery	Devices advertise _arduino._tcp + secureota=true TXT record. IDE discovers devices via mDNS exactly as in stock Arduino IDE.
OTA Channel	Encrypted TCP (TLS 1.3) on port 3232. AES-256 encrypted, HMAC-tagged firmware stream.
Status Channel	UDP broadcast on port 5007 (device status packets consumed by Device Dashboard).

5.3 Data Flow Summary

Flow	Source	Destination	Protocol	Data	Security
Compile	IDE (Arduino CLI)	Local .bin file	Local filesystem	Compiled firmware binary	N/A — local

Flow	Source	Destination	Protocol	Data	Security
Discovery	ESP32	IDE Device Dashboard	mDNS / UDP	Device name, IP, secureota flag	Local network
Status	ESP32	IDE Device Dashboard	UDP Broadcast	Device name, firmware version, IP	Local network
Key Delivery	IDE Upload Engine	ESP32 OTA Library	TLS 1.3 TCP 3232 (ECDH handshake)	Encrypted AES session key	ECDH forward secrecy
Firmware OTA	IDE Upload Engine	ESP32 OTA Library	TLS 1.3 TCP 3232	AES-256 encrypted firmware chunks + HMAC tags	AES-256 + HMAC-SHA256 + TLS
Metadata	IDE Upload Engine	ESP32 OTA Library	TLS 1.3 TCP 3232	SHA-256 hash, ECDSA signature, version, size	TLS 1.3 + ECDSA
Version Log	IDE Upload Engine	Version History Panel	Local DB write	Sketch name, version, device, timestamp, result	Local filesystem

6. Proposed System Explanation

6.1 Comparision: Standar Arduino OTA Vs Secureota IDE

Feature	Standard Arduino OTA	SecureOTA IDE (Proposed)
Firmware Encryption	None — .bin sent in plaintext	AES-256-CBC encryption before transmission
Transport Security	Plain TCP/UDP (no TLS)	TLS 1.3 with mutual certificate authentication
Firmware Authentication	None — any sender accepted	ECDSA digital signature verified on device
Integrity Check	None	SHA-256 end-to-end hash + HMAC-SHA256 per chunk
Device Authentication	Password only (optional, plaintext)	Certificate-based mTLS — only authorised IDE can upload
Firmware Version Tracking	None — IDE logs nothing	Built-in Version History Panel with full upload record
Update Relevance Check	None — all sketches treated equally	Relevance Checker scores sketch vs. device function tags
Device Dashboard	No built-in dashboard	Embedded Device Dashboard panel with live device status
Rollback Support	Manual re-upload required	One-click rollback from Version History Panel
User Experience	One-click Upload (Network Port)	Identical one-click Upload — no extra steps for user

6.2 Step-By-Step: First-Time Device Provisioning

Before encrypted OTA can be used, a one-time provisioning step is required via USB Serial (same as initial Arduino sketch upload):

1. User opens SecureOTA IDE and navigates to Tools > SecureOTA > Provision Device.

2. IDE generates an ECDSA key pair for the target device and stores the private key in the device's NVS (Non-Volatile Storage) flash partition.
3. The IDE's own ECDSA public key is embedded into the device's firmware image and written to a dedicated key partition.
4. The modified SecureArduinoOTA library is flashed to the device via USB Serial, along with the device's key pair and the IDE public key.
5. Provisioning complete. From this point, all OTA uploads from the IDE are fully encrypted, authenticated, and tracked.

6.2.1 Step-by-Step: Secure OTA Upload (After Provisioning)

6. User writes/modifies sketch in the SecureOTA IDE editor (identical to Arduino IDE).
7. User assigns a version tag (e.g., v1.3.0) and function_tags (e.g., temperature_sensor, wifi_reporting) in the Version Tag toolbar.
8. User selects the target device from the Port menu (devices discovered via mDNS, same as stock Arduino OTA).
9. User clicks Upload. The Relevance Checker panel appears, showing the relevance score of this sketch against the device's function profile. User confirms or cancels.
10. The IDE compiles the sketch via Arduino CLI (unchanged). The resulting .bin file is passed to the SecureOTA Upload Engine.
11. Upload Engine computes SHA-256 of the .bin, generates ECDSA signature, generates AES-256 session key + IV.
12. Upload Engine establishes TLS 1.3 TCP connection to device on port 3232.
13. Encrypted session key (ECDH), metadata (hash, signature, version, size), and AES-256 encrypted firmware chunks are transmitted.
14. Device's SecureArduinoOTA library receives, verifies HMAC per chunk, decrypts firmware, verifies hash and signature.
15. Device writes verified firmware to standby partition and reboots. New firmware broadcasts updated version string.
16. IDE Version History Panel logs the upload. Device Dashboard updates to reflect new firmware version.

7. Number of Modules

SecureOTA IDE is composed of eight modules. Modules 1–3 exist in the stock Arduino IDE (unmodified); Modules 4–8 are new additions developed as part of this project.

Module ID	Module Name	Location	Language	Status
MOD-01	Arduino IDE Core (Editor, Compiler, Board Manager)	IDE Host	TypeScript, Go (Arduino CLI)	Unchanged — cloned from Arduino IDE 2.x
MOD-02	Network Port Discovery (mDNS)	IDE Host	TypeScript	Unchanged — reused from stock IDE
MOD-03	ArduinoOTA Base Library	ESP32 Device	C++ (Arduino)	Extended — base for MOD-08
MOD-04	SecureOTA Upload Engine	IDE Host	TypeScript / Node.js crypto	NEW
MOD-05	Firmware Version History Panel	IDE Host	TypeScript + React (Theia Extension)	NEW
MOD-06	Update Relevance Checker Panel	IDE Host	TypeScript + React (Theia Extension)	NEW
MOD-07	Device Dashboard Panel	IDE Host	TypeScript + React (Theia Extension)	NEW
MOD-08	SecureArduinoOTA Library (Modified)	ESP32 Device	C++ + mbedTLS	NEW (extends MOD-03)

8. Functionality of Modules

MOD-01: Arduino IDE Core

- Provides the code editor (Monaco editor via Theia), syntax highlighting, auto-completion, and library manager — all identical to Arduino IDE 2.x.
- Invokes Arduino CLI for sketch compilation, board package management, and USB Serial upload.
- Manages board definitions, port enumeration (Serial + Network), and sketch project files.
- This module is cloned verbatim and forms the foundation of Secure OTA IDE. No changes are made to core editor or compilation logic.

MOD-02: Network Port Discovery (mDNS)

- Performs mDNS queries for `_arduino.tcp` service records on the local network.
- Extended minimally to also read the `secure Ota=true` TXT record from discovered devices, flagging them as Secure OTA-capable in the Port menu.
- Passes device IP, hostname, and port to the upload pipeline when a network port is selected.

MOD-03: ArduinoOTA Base Library

- Original open-source C++ library that provides OTA update reception on ESP32/Arduino devices.
- Handles mDNS advertisement (`_arduino._tcp`), TCP socket listening on port 3232, and raw binary reception.
- Directly extended by MOD-08 — all base functionality is preserved; encryption layer is added on top.

MOD-04: SecureOTA Upload Engine

- Intercepts the upload command when a SecureOTA-capable network port is selected.
- Reads the compiled `.bin` file path from Arduino CLI output.
- Computes SHA-256 hash of the `.bin` using Node.js crypto module.
- Generates an ECDSA-P256 signature over the SHA-256 digest using the IDE's stored private key.
- Generates a fresh AES-256-CBC session key and 128-bit IV per upload.

- Packages: [AES-encrypted .bin | SHA-256 hash | ECDSA signature | version metadata | per-chunk HMAC tags].
- Establishes TLS 1.3 TCP connection to the device IP on port 3232.
- Transmits the encrypted package in 4 KB chunks, each with an HMAC-SHA256 tag.
- Receives upload result acknowledgement from device and reports SUCCESS or FAILED to the IDE console.
- On completion, writes upload record to the Version History database (triggers MOD-05 refresh).

MOD-05: Firmware Version History Panel

- A Theia side-panel extension displaying a table of all past OTA uploads from this IDE instance.
- Persists upload records in a local SQLite database on the host machine (sketch name, version tag, function tags, device IP/name, timestamp, result, SHA-256 hash of binary).
- Supports one-click rollback: re-transmits a selected historical binary through MOD-04's secure upload pipeline.
- Supports export of the full history log as a CSV file.
- Displays a diff-style version comparison (version tags and function tags) between any two selected entries.

MOD-06: Update Relevance Checker Panel

- Triggered automatically before each OTA upload when the target device has a registered function profile.
- Reads the sketch's function_tags from the Version Tag toolbar and the device's function_profile from the device registry (stored locally during provisioning).
- Computes the relevance score using the tag intersection algorithm (Section 4.7).
- Displays matched tags (green), unmatched tags (yellow), and missing tags (red) in a visual breakdown.
- Shows a relevance score percentage and a recommendation: PROCEED / WARN / STRONG WARNING.
- Allows the user to override and proceed, or cancel the upload.

MOD-07: Device Dashboard Panel

- A persistent Theia side-panel that listens for UDP status broadcasts from SecureOTA devices on the local network (port 5007).
- Displays a live table of all discovered devices: device name, IP address, current firmware version, last-updated timestamp, online/offline status.
- Provides a direct Upload to Device button per row, which pre-selects that device as the target port for the next upload.
- Updates in real time when a device broadcasts a new firmware version after a successful OTA update.
- Marks a device as offline if no status broadcast is received within 90 seconds.

MOD-08: Secure ArduinoOTA Library (Modified Device Library)

- Drop-in replacement for the standard ArduinoOTA library — same API (begin(), handle()), same sketch include pattern (SecureArduinoOTA.h).
- Advertises _arduino._tcp mDNS with additional TXT record: secureota=true, allowing IDE to distinguish this device.
- Listens on TCP port 3232 with TLS 1.3 enabled using ESP32 mbedTLS (certificate loaded from device NVS at runtime).
- On connection: completes TLS handshake, receives ECDH-wrapped AES session key, decrypts it using device private key from NVS.
- Receives firmware chunks; verifies HMAC-SHA256 for each chunk before buffering to standby OTA partition.
- After full receipt: AES-256-CBC decrypts assembled firmware, verifies SHA-256 hash against transmitted hash, verifies ECDSA signature against embedded IDE public key.
- On all checks passing: writes to standby partition, calls esp_ota_set_boot_partition(), triggers restart.
- On any failure: clears standby partition buffer, sends NACK to IDE, continues running on current firmware.
- Broadcasts UDP status packet every 30 seconds on port 5007: device name, firmware version string, IP address.

9. Test Case Technique

9.1: Testing Strategies

9.1.1 Black-Box Testing

Applied to all IDE panel interactions (Version History Panel, Relevance Checker, Device Dashboard) and the end-to-end OTA upload flow. Test cases are derived directly from functional requirements (FR-01 through FR-12). The tester interacts with the IDE exactly as an end-user would, without knowledge of internal implementation.

9.1.2 White-Box Testing

Applied to MOD-04 (SecureOTA Upload Engine) and MOD-08 (SecureArduinoOTA Library) where cryptographic logic paths must be explicitly validated. Branch coverage targets include: hash match / mismatch, signature valid / invalid, HMAC pass / fail, TLS handshake success / failure, and all rollback trigger conditions.

9.1.3 Boundary Value Analysis (BVA)

Applied to: firmware binary size boundaries (minimum 1-byte sketch, maximum ESP32 partition size ~1.5 MB), chunk size boundaries (1-byte chunk, 4096-byte chunk, exactly partition-size firmware), relevance score boundaries (0.0, 0.30 threshold, 1.0), and version tag string length limits.

9.1.4 Equivalence Partitioning

Applied to device authentication (valid TLS cert, expired cert, no cert, revoked cert), firmware integrity (valid hash + valid sig, valid hash + invalid sig, invalid hash + valid sig, both invalid), and upload results (SUCCESS, FAILED at transfer, FAILED at verification, ROLLBACK).

9.1.5 Fault Injection Testing

Deliberately introduced faults to validate robustness: Wi-Fi disconnection at 50% firmware transfer, single corrupted byte in a chunk, wrong device IP selected, device powered off mid-upload, and boot failure after flashing. In every case, the expected outcome is that the device continues running its previous firmware with no data corruption.

9.2 Test Levels

- Unit Testing: MOD-04 crypto functions (encrypt, sign, hash) tested in isolation with known test vectors. MOD-06 relevance algorithm tested with a matrix of tag combinations.
- Integration Testing: MOD-04 + MOD-08 tested together over a local loopback for the full encrypt → transmit → decrypt → verify pipeline.
- System Testing: Full IDE-to-ESP32 OTA upload with real hardware on a Wi-Fi network. All panels verified for correct data.
- Security / Penetration Testing: Network capture analysis (Wireshark) to confirm firmware ciphertext is visible but not decryptable. Forged firmware injection attempt. Replay of previous valid upload package.
- Regression Testing: All standard Arduino IDE features (USB upload, Serial Monitor, Board Manager, Library Manager) tested after each development increment to confirm no regression.

10. Test Case Log

10.1 Functional Test Cases

TC ID	Module	Test Description	Input / Action	Expected Output
TC-F-01	MOD-01	Sketch compiles identically to stock Arduino IDE	Open existing sketch, click Verify	Compilation succeeds; .bin identical to stock Arduino IDE output (same SHA-256)
TC-F-02	MOD-02	SecureOTA device discovered in Port menu	ESP32 running SecureArduino OTA powered on Wi-Fi	Device appears in Port menu with secureota flag; name shown correctly
TC-F-03	MOD-04	Firmware encrypted before transmission	Select network port; click Upload	Wireshark capture on port 3232 shows ciphertext — no readable .bin bytes visible
TC-F-04	MOD-04	TLS 1.3 channel established for OTA	Click Upload with SecureOTA network port selected	TLS 1.3 handshake completes; certificate verified; data flows on encrypted channel
TC-F-05	MOD-04	ECDSA signature generated and attached	Upload triggered with IDE private key available	Metadata package contains valid ECDSA-P256 signature over SHA-256 of firmware
TC-F-06	MOD-08	Device decrypts and verifies firmware correctly	IDE sends valid encrypted + signed firmware	Device decrypts, hash matches, signature valid; firmware written to standby partition

TC ID	Module	Test Description	Input / Action	Expected Output
TC-F-07	MOD-08	Device rejects tampered firmware	1 byte in encrypted firmware modified in transit (simulated)	HMAC verification fails at chunk 12; device aborts; stays on current firmware
TC-F-08	MOD-08	Device rejects firmware with wrong signature	Firmware signed with unknown key	ECDSA verification fails; esp_ota_set_boot_partition() not called; NACK sent to IDE
TC-F-09	MOD-05	Version History Panel records successful upload	Successful OTA upload of v1.2.0 to esp32_sensor	History panel shows new entry: sketch name, v1.2.0, device IP, timestamp, SUCCESS
TC-F-10	MOD-05	Rollback re-uploads previous version	User clicks Rollback to v1.1.0 in History panel	v1.1.0 binary re-uploaded via secure channel; device boots v1.1.0; history shows ROLLBACK entry
TC-F-11	MOD-06	Relevance Checker scores matching tags correctly	Sketch tags: [temperature, wifi]; Device profile: [temperature, sensor]	Score = 0.5 (1 of 2 device tags matched); panel shows PROCEED with yellow warning
TC-F-12	MOD-06	Relevance Checker warns on zero match	Sketch tags: [display, ui]; Device profile: [temperature, sensor]	Score = 0.0; STRONG WARNING displayed; upload proceeds only if user explicitly overrides

TC ID	Module	Test Description	Input / Action	Expected Output
TC-F-13	MOD-07	Device Dashboard shows live device status	ESP32 broadcasting status packets on UDP 5007	Dashboard table shows correct device name, IP, firmware version, and ONLINE status
TC-F-14	MOD-07	Dashboard marks device offline after timeout	Stop UDP broadcasts from device for 90 seconds	Dashboard marks device as OFFLINE after 90-second timeout

10.2 Security Test Cases

TC ID	Attack Scenario	Test Action	Expected Result
TC-S-01	Plaintext firmware capture	Capture TCP traffic on port 3232 during OTA with Wireshark	Only TLS ciphertext visible; no readable firmware bytes
TC-S-02	Firmware injection (MITM)	Intercept TLS channel; attempt to inject different firmware binary	TLS certificate mismatch or HMAC failure; device rejects injected data
TC-S-03	Forged firmware (no valid IDE signature)	Upload firmware signed with a self-generated non-IDE key	ECDSA verification fails on device; firmware not written to flash
TC-S-04	Replay attack (old valid package)	Re-send a previously captured valid OTA session to the device	Device rejects re-upload of same version (version check); no re-flash

TC ID	Attack / Scenario	Test Action	Expected Result
TC-S-05	Unauthorised IDE upload attempt	Connect different (non-provisioned) IDE to provisioned device	TLS handshake fails — IDE's certificate not in device's trust store
TC-S-06	Corrupted AES session key	Modify 1 byte of the transmitted encrypted session key	Device cannot decrypt session key; decryption fails; firmware not written
TC-S-07	SHA-256 hash mismatch	Transmit valid firmware but with incorrect hash in metadata	Device decrypts correctly but computed hash != transmitted hash; update aborted

10.3 Fault Tolerance Test Cases

TC ID	Fault Scenario	Test Action	Expected Result
TC-FT-01	Wi-Fi drop at 50% transfer	Disable Wi-Fi on development machine mid-upload	TLS connection drops; device detects incomplete transfer; standby partition cleared; device stays on current firmware
TC-FT-02	Device powered off mid-upload	Cut power to ESP32 at 70% transfer	On reboot, device finds incomplete standby partition; remains on current (previous) firmware

TC ID	Fault Scenario	Test Action	Expected Result
TC-FT-03	Boot failure after flash	Flash a valid but non-bootable binary (valid sig, broken application code)	Device fails to call app_main() within watchdog window; ESP-IDF OTA manager rolls back to previous partition
TC-FT-04	Duplicate upload (same version)	Upload same version already running on device	Version check in device library detects same version; update skipped; IDE shows VERSION_CURRENT
TC-FT-05	IDE crash during upload	Force-quit IDE during chunk transmission	Device detects TLS connection closed unexpectedly; clears partial buffer; stays on current firmware

10.4 Regression Test Cases (Arduino IDE Compatibility)

TC ID	Feature Tested	Expected Result
TC-R-01	USB Serial upload (standard Arduino)	Sketch uploads via COM/USB port identically to stock Arduino IDE
TC-R-02	Serial Monitor	Serial Monitor opens and displays device output correctly
TC-R-03	Board Manager	ESP32, Arduino Uno, and Nano board packages install and update correctly
TC-R-04	Library Manager	Libraries install, update, and include in sketches correctly
TC-R-05	Standard (unencrypted) ArduinoOTA library	Existing sketches using the original ArduinoOTA.h still compile; standard OTA still functions for non-SecureOTA devices

10.5 Test Summary

Category	Total
Functional	14
Security	7
Fault Tolerance	5
Regression	5
TOTAL	31

References

- Arduino LLC. (2024). Arduino IDE 2 Source Code. GitHub: [arduino/arduino-ide](https://github.com/arduino/arduino-ide). <https://github.com/arduino/arduino-ide>
- Arduino LLC. (2024). ArduinoOTA Library Source Code. GitHub: [jandrassy/ArduinoOTA](https://github.com/jandrassy/ArduinoOTA).
- Eclipse Foundation. (2024). Theia IDE Framework Documentation. <https://theia-ide.org/docs/>
- Espressif Systems. (2024). ESP-IDF Programming Guide — OTA Updates and mbedTLS. <https://docs.espressif.com/projects/esp-idf/>
- National Institute of Standards and Technology. FIPS 197 — AES (2001) and FIPS 186-5 — DSS (2023).
- Rescorla, E. (2018). The Transport Layer Security (TLS) Protocol Version 1.3. RFC 8446. IETF.
- Node.js Foundation. (2024). Crypto Module Documentation. <https://nodejs.org/api/crypto.html>